

CleanRobot_STU - Team Guide

- [CleanRobot_STU — Tài liệu làm việc nhóm](#)
 - [0. Mục tiêu tài liệu](#)
 - [1. Kiến trúc tổng quan](#)
 - [2. Cấu trúc workspace](#)
 - [3. Quy tắc chia package](#)
 - [3.1. clean_robot_msgs](#)
 - [3.2. clean_robot_description](#)
 - [3.3. clean_robot_perception](#)
 - [3.4. clean_robot_mission](#)
 - [3.5. clean_robot_navigation](#)
 - [3.6. clean_robot_base](#)
 - [3.7. clean_robot_simulation](#)
 - [3.8. clean_robot_bringup](#)
 - [4. Contract giao tiếp giữa các team](#)
 - [4.1. Topic contract](#)
 - [4.2. Action contract](#)
 - [4.3. Service contract gợi ý](#)
 - [5. Ai làm gì để song song?](#)
 - [Team A — Simulation](#)
 - [Team B — Perception](#)
 - [Team C — Mission](#)
 - [Team D — Navigation](#)
 - [Team E — Base / Hardware](#)
 - [6. Thứ tự tích hợp](#)
 - [Giai đoạn 1 — Workspace và message](#)
 - [Giai đoạn 2 — Simulation/Base chạy được](#)
 - [Giai đoạn 3 — Nav2 chạy được](#)
 - [Giai đoạn 4 — Mission chạy với fake target](#)
 - [Giai đoạn 5 — Perception thật](#)
 - [Giai đoạn 6 — Full MVP](#)
 - [7. Quy tắc Git cho team](#)
 - [Branch](#)
 - [Commit message](#)
 - [8. Quy tắc code](#)
 - [Naming](#)
 - [Logging](#)
 - [Không làm](#)

- [9. .gitignore gợi ý](#)
- [10. Checklist trước khi merge](#)
- [11. Lệnh cài đặt cơ bản](#)
- [12. Build workspace](#)
- [13. Chạy từng phần để debug](#)
 - [Kiểm tra message](#)
 - [Kiểm tra mission state](#)
 - [Kiểm tra target](#)
 - [Kiểm tra Nav2 action](#)
 - [Kiểm tra lệnh điều khiển xe](#)
- [14. Roadmap MVP](#)
 - [Sprint 1](#)
 - [Sprint 2](#)
 - [Sprint 3](#)
 - [Sprint 4](#)
 - [Sprint 5](#)
 - [Sprint 6](#)
- [15. Định nghĩa hoàn thành MVP](#)

CleanRobot_STU — Tài liệu làm việc nhóm

0. Mục tiêu tài liệu

Tài liệu này dùng để nhiều người có thể làm việc song song trên dự án **CleanRobot_STU** mà không giẫm chân nhau.

Mục tiêu MVP hiện tại:

Robot tìm thấy rác → tính vị trí rác → chạy tới rác → dừng.

Chưa làm:

- Nhặt rác
- Thả rác
- Phân loại rác nâng cao
- Tối ưu exploration phức tạp

Nguyên tắc thiết kế:

Mỗi package phụ trách một nhóm chức năng.
Mỗi node chỉ làm một việc rõ ràng.
Các node giao tiếp qua topic / service / action.
Không để YOLO điều khiển robot trực tiếp.

1. Kiến trúc tổng quan

Flow MVP:

```
Camera
↓
YOLO nhận diện rác
↓
Trash Localization tính vị trí rác trong map
↓
Target Manager chọn rác phù hợp
↓
Mission Manager gửi goal cho Nav2
↓
Nav2 tính đường và publish /cmd_vel
↓
Motor Controller hoặc Webots Driver
↓
Robot chạy tới rác
↓
Dừng robot và báo đã tới rác
```

Khi chưa thấy rác:

```
Mission Manager
↓
Search Behavior
↓
Nav2 đi qua waypoint tìm kiếm
↓
Camera + YOLO tiếp tục quét
```

Khi thấy rác:

```
Target Manager chọn target
↓
Mission Manager hủy goal tìm kiếm nếu có
↓
Mission Manager gửi goal mới tới Nav2
↓
Robot chạy tới rác
```

2. Cấu trúc workspace

```
clean_robot_ws/  
└─ src/  
    ├── clean_robot_msgs/  
    ├── clean_robot_description/  
    ├── clean_robot_perception/  
    ├── clean_robot_mission/  
    ├── clean_robot_navigation/  
    ├── clean_robot_base/  
    ├── clean_robot_simulation/  
    └─ clean_robot_bringup/
```

Không phải mọi package ROS2 đều nằm trong `src/`.

Các package tự code của team nằm trong `clean_robot_ws/src/`.

Các package cài bằng apt như Nav2, Webots ROS2, slam_toolbox thường nằm trong `/opt/ros/jazzy/`.

3. Quy tắc chia package

3.1. `clean_robot_msgs`

Vai trò:

Định nghĩa message dùng chung giữa các package.

Package này là hợp đồng dữ liệu. Ai làm perception, mission, navigation đều phải tuân theo message ở đây.

Cấu trúc đề xuất:

```
clean_robot_msgs/  
├─ package.xml  
├─ CMakeLists.txt  
└─ msg/  
    ├── TrashDetection2D.msg  
    ├── TrashDetection2DArray.msg  
    ├── TrashCandidate.msg  
    └─ TrashCandidateArray.msg
```

TrashDetection2D.msg

Dùng cho kết quả YOLO trên ảnh 2D.

```
std_msgs/Header header
string class_name
float32 confidence
int32 x
int32 y
int32 width
int32 height
```

Ý nghĩa:

YOLO thấy một vật nghi là rác tại vùng bbox trên ảnh.

TrashDetection2DArray.msg

```
std_msgs/Header header
TrashDetection2D[] detections
```

TrashCandidate.msg

Dùng cho rác đã được chuyển từ ảnh sang tọa độ robot/map.

```
std_msgs/Header header
string id
string class_name
float32 confidence
geometry_msgs/PoseStamped pose
bool reachable
string status
```

Ý nghĩa:

Rác có tọa độ thật, có thể gửi cho Nav2 đi tới.

TrashCandidateArray.msg

```
std_msgs/Header header
TrashCandidate[] candidates
```

Người phụ trách package này cần làm:

- Tạo msg
- Build kiểm tra message sinh ra đúng
- Không tự ý đổi message khi các package khác đã dùng

Lệnh kiểm tra:

```
ros2 interface show clean_robot_msgs/msg/TrashCandidate
```

3.2. clean_robot_description

Vai trò:

Mô tả cơ thể robot: thân xe, bánh xe, camera, lidar, imu, frame TF.

Cấu trúc đề xuất:

```
clean_robot_description/  
├─ package.xml  
├─ CMakeLists.txt  
├─ urdf/  
│   ├── clean_robot.urdf.xacro  
│   ├── base.urdf.xacro  
│   ├── wheels.urdf.xacro  
│   └─ sensors.urdf.xacro  
├─ meshes/  
├─ rviz/  
│   └─ robot_view.rviz  
└─ launch/  
    └─ display.launch.py
```

Package này trả lời:

- base_link nằm ở đâu?
- camera_link nằm ở đâu?
- lidar_link nằm ở đâu?
- imu_link nằm ở đâu?
- bánh trái/phải nằm ở đâu?

Frame TF đề xuất:

```
map  
├─ odom  
│   └─ base_link  
│       ├── base_footprint  
│       ├── camera_link  
│       │   └─ camera_optical_frame  
│       ├── lidar_link  
│       ├── imu_link  
│       ├── left_wheel_link  
│       └─ right_wheel_link
```

Người phụ trách package này cần làm:

- Tạo URDF/Xacro tối thiểu
- Chạy robot_state_publisher
- Kiểm tra robot hiển thị đúng trong RViz
- Kiểm tra TF không thiếu frame

Lệnh kiểm tra:

```
ros2 launch clean_robot_description display.launch.py
ros2 run tf2_tools view_frames
```

3.3. clean_robot_perception

Vai trò:

Nhìn ảnh, phát hiện rác, tính vị trí rác.

Cấu trúc đề xuất:

```
clean_robot_perception/
├── package.xml
├── setup.py
├── setup.cfg
├── resource/
│   └── clean_robot_perception
├── clean_robot_perception/
│   ├── __init__.py
│   ├── trash_detector_node.py
│   ├── trash_localization_node.py
│   ├── trash_tracker_node.py
│   ├── fake_trash_candidate_node.py
│   ├── yolo_model.py
│   └── image_utils.py
```

trash_detector_node.py

Nhiệm vụ:

/camera/image_raw → YOLO → /trash/detections

Subscribe:

/camera/image_raw

Publish:

/trash/detections

Type:

clean_robot_msgs/msg/TrashDetection2DArray

Không được làm:

- Không publish /cmd_vel
- Không gửi goal Nav2
- Không quyết định robot đi đâu

trash_localization_node.py

Nhiệm vụ:

```
/trash/detections + depth/lidar/tf → /trash/candidates
```

Subscribe có thể gồm:

```
/trash/detections  
/camera/depth/image_raw  
/scan  
/tf
```

Publish:

```
/trash/candidates
```

Type:

```
clean_robot_msgs/msg/TrashCandidateArray
```

trash_tracker_node.py

Nhiệm vụ sau này:

```
Lọc detection nhiễu, tránh thấy 1 frame rồi chạy ngay.
```

Rule gợi ý:

```
Chỉ chấp nhận rác nếu xuất hiện liên tục ít nhất 3 frame.  
Nếu mất detection quá 5 giây thì xóa candidate.
```

fake_trash_candidate_node.py

Nhiệm vụ:

```
Giả lập rác để test mission trước khi YOLO thật hoàn thành.
```

Publish:

```
/trash/candidates
```

Người phụ trách package này cần làm:

- Đảm bảo publish đúng message contract
- Có fake node để test độc lập
- Không điều khiển robot trực tiếp

Lệnh kiểm tra:

```
ros2 topic echo /trash/detections
ros2 topic echo /trash/candidates
```

3.4. clean_robot_mission

Vai trò:

Điều phối nhiệm vụ: tìm rác, chọn rác, gửi goal cho Nav2, xử lý kết quả.

Cấu trúc đề xuất:

```
clean_robot_mission/
├── package.xml
├── setup.py
├── setup.cfg
├── resource/
│   └── clean_robot_mission
├── clean_robot_mission/
│   ├── __init__.py
│   ├── mission_manager_node.py
│   ├── target_manager_node.py
│   ├── search_behavior.py
│   ├── state_machine.py
│   ├── nav2_client.py
│   └── target_store.py
```

target_manager_node.py

Nhiệm vụ:

/trash/candidates → chọn target tốt nhất → /trash/target_pose

Subscribe:

/trash/candidates

Publish:

/trash/target_pose
/trash/target_id

Logic MVP:

Chọn candidate có confidence cao nhất và reachable=true.

Logic nâng cấp:

```
score = confidence * 0.7 + distance_score * 0.3
```

Không chọn target đã failed nhiều lần.
Không đổi target liên tục nếu target hiện tại vẫn hợp lệ.

mission_manager_node.py

Nhiệm vụ:

Nếu chưa có rác → gọi search_behavior để đi tìm.
Nếu có target → gửi NavigateToPose cho Nav2.
Nếu tới rác → chuyển REACHED_TRASH.
Nếu fail → quay lại SEARCHING.

Subscribe:

/trash/target_pose

Publish:

/mission/state

Action client:

/navigate_to_pose

search_behavior.py

Nhiệm vụ:

Chọn cách robot đi tìm rác khi chưa thấy rác.

Mode MVP:

waypoint_patrol

Ví dụ waypoint:

```
search_waypoints:
- [0.0, 0.0, 0.0]
- [1.0, 0.0, 0.0]
- [1.0, 1.0, 1.57]
- [0.0, 1.0, 3.14]
```

state_machine.py

State đề xuất:

```
IDLE
SEARCHING
NAVIGATING_SEARCH
NAVIGATING_TO_TRASH
REACHED_TRASH
FAILED
```

Transition:

```
IDLE → SEARCHING
SEARCHING → NAVIGATING_SEARCH
SEARCHING → NAVIGATING_TO_TRASH
NAVIGATING_SEARCH → SEARCHING
NAVIGATING_TO_TRASH → REACHED_TRASH
NAVIGATING_TO_TRASH → SEARCHING nếu fail
```

nav2_client.py

Nhiệm vụ:

Bọc logic gửi goal, hủy goal, đọc result từ Nav2.

Người phụ trách package này cần làm:

- Không xử lý YOLO
- Không xử lý motor trực tiếp
- Chỉ điều phối qua topic/action
- Có log rõ state hiện tại

Lệnh kiểm tra:

```
ros2 topic echo /mission/state
ros2 topic echo /trash/target_pose
ros2 action list | grep navigate
```

3.5. clean_robot_navigation

Vai trò:

Chứa config và launch cho Nav2, SLAM, AMCL, map.

Cấu trúc đề xuất:

```

clean_robot_navigation/
├── package.xml
├── config/
│   ├── nav2_params.yaml
│   ├── slam_toolbox_params.yaml
│   └── amcl_params.yaml
├── maps/
│   ├── clean_room.yaml
│   └── clean_room.pgm
└── launch/
    ├── nav2.launch.py
    ├── slam.launch.py
    └── localization.launch.py

```

Package này không cần tự code nhiều node. Nó chủ yếu chứa cấu hình.

Input chính cho Nav2:

```

/map
/scan
/odom
/tf
goal pose

```

Output chính:

```

/cmd_vel

```

Người phụ trách package này cần làm:

- Chạy SLAM khi chưa có map
- Chạy AMCL khi đã có map
- Cấu hình Nav2 planner/controller/costmap
- Test click goal trong RViz robot chạy được

Lệnh kiểm tra:

```

ros2 launch clean_robot_navigation nav2.launch.py
ros2 topic echo /cmd_vel

```

Tiêu chí hoàn thành:

RViz click một điểm → robot tự chạy tới điểm đó.

Nếu tiêu chí này chưa đạt, đừng vội gắn YOLO vào.

3.6. clean_robot_base

Vai trò:

Điều khiển xe thật.

Nếu chỉ chạy Webots, package này có thể làm sau.

Cấu trúc đề xuất:

```
clean_robot_base/  
├─ package.xml  
├─ setup.py  
├─ setup.cfg  
├─ resource/  
│   └─ clean_robot_base  
└─ clean_robot_base/  
    ├─ __init__.py  
    ├─ motor_controller_node.py  
    ├─ encoder_node.py  
    ├─ odom_publisher_node.py  
    ├─ serial_driver.py  
    └─ kinematics.py
```

motor_controller_node.py

Subscribe:

/cmd_vel

Gửi xuống:

Arduino / ESP32 / motor driver

Nhiệm vụ:

/cmd_vel → tốc độ bánh trái/phải → phần cứng.

encoder_node.py

Publish:

/wheel/odom

kinematics.py

Chứa công thức xe vi sai:

linear.x, angular.z → left_wheel_speed, right_wheel_speed

Người phụ trách package này cần làm:

- Robot nhận /cmd_vel và chạy đúng hướng
- Publish odom ổn định
- Có emergency stop nếu mất kết nối

Lệnh kiểm tra:

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.1}, angular: {z: 0.0}}"
```

3.7. clean_robot_simulation

Vai trò:

Chạy robot trong Webots.

Cấu trúc đề xuất:

```
clean_robot_simulation/
├── package.xml
├── worlds/
│   └── clean_room.wbt
├── protos/
│   └── CleanRobot.proto
├── launch/
│   └── webots_sim.launch.py
├── config/
│   └── webots_bridge.yaml
```

Package này chứa:

- World Webots
- Robot mô phỏng
- Sensor mô phỏng
- Bridge Webots ↔ ROS2

Không chứa:

- YOLO
- Mission logic
- Nav2 config chính

Người phụ trách package này cần làm:

- Mở Webots được bằng launch
- Robot có camera/lidar/odom/cmd_vel
- Topic ROS2 xuất hiện đúng

Lệnh kiểm tra:

```
ros2 topic list
ros2 topic echo /camera/image_raw
ros2 topic echo /scan
ros2 topic echo /odom
```

3.8. clean_robot_bringup

Vai trò:

Launch toàn hệ thống.

Cấu trúc đề xuất:

```
clean_robot_bringup/
├── package.xml
├── setup.py
├── setup.cfg
├── resource/
│   └── clean_robot_bringup
├── launch/
│   ├── find_and_go_sim.launch.py
│   ├── find_and_go.launch.py
│   └── full_system.launch.py
└── config/
    ├── mission_params.yaml
    ├── perception_params.yaml
    └── robot_params.yaml
```

find_and_go_sim.launch.py

Chạy bản mô phỏng:

```
Webots
robot_state_publisher
Nav2
fake_trash_candidate_node hoặc trash_detector_node
target_manager_node
mission_manager_node
```

find_and_go.launch.py

Chạy bản robot thật:

```
Camera driver
LiDAR driver
robot_state_publisher
Nav2
perception
mission
base
```

Người phụ trách package này cần làm:

- Gom launch file
- Không viết logic xử lý chính trong bringup
- Chỉ cấu hình cách chạy hệ thống

4. Contract giao tiếp giữa các team

4.1. Topic contract

Topic	Type	Publisher	Subscriber	Mục đích
/camera/ image_raw	sensor_msgs/msg/Image	Camera/Webots	Perception	Ảnh đầu vào
/trash/ detections	clean_robot_msgs/msg/ TrashDetection2DArray	Detector	Localization	Kết quả YOLO 2D
/trash/ candidates	clean_robot_msgs/msg/ TrashCandidateArray	Localization/Fake	Target Manager	Rác có tọa độ map
/trash/ target_pose	geometry_msgs/msg/ PoseStamped	Target Manager	Mission Manager	Vị trí rác được chọn
/trash/ target_id	std_msgs/msg/String	Target Manager	Mission/Debug	ID target
/mission/state	std_msgs/msg/String	Mission Manager	UI/Debug	State hiện tại
/scan	sensor_msgs/msg/ LaserScan	LiDAR/Webots	Nav2/SLAM	Tránh vật cản
/odom	nav_msgs/msg/Odometry	Base/Webots	Nav2/SLAM	Odometry
/tf	tf2_msgs/msg/ TFMessage	Many	Many	Frame transform
/cmd_vel	geometry_msgs/msg/ Twist	Nav2	Base/Webots	Lệnh vận tốc

4.2. Action contract

Action	Type	Client	Server	Mục đích
/ navigate_to_pose	nav2_msgs/ action/ NavigateToPose	Mission Manager	Nav2	Đi tới waypoint hoặc rác

4.3. Service contract gợi ý

Có thể thêm sau:

Service	Mục đích
/start_mission	Bắt đầu tìm rác
/stop_mission	Dừng nhiệm vụ
/reset_targets	Xóa target đã lưu
/clear_costmap	Reset costmap Nav2

5. Ai làm gì để song song?

Team A — Simulation

Phụ trách:

```
clean_robot_simulation
clean_robot_description phần mô phỏng nếu cần
```

Đầu ra bắt buộc:

```
/camera/image_raw
/scan
/odom
/tf
Robot nhận /cmd_vel và chạy trong Webots
```

Có thể test độc lập bằng:

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.1}, angular: {z: 0.0}}"
```

Team B — Perception

Phụ trách:

```
clean_robot_perception
```

Đầu ra bắt buộc:

```
/trash/detections
/trash/candidates
```

Có thể test độc lập bằng ảnh/camera giả.

Không cần chờ Nav2 hoàn thành.

Team C — Mission

Phụ trách:

```
clean_robot_mission
```

Đầu vào bắt buộc:

```
/trash/candidates hoặc /trash/target_pose
```

Đầu ra bắt buộc:

```
/mission/state  
Nav2 goal qua /navigate_to_pose
```

Có thể test độc lập bằng `fake_trash_candidate_node`.

Team D — Navigation

Phụ trách:

```
clean_robot_navigation
```

Đầu vào bắt buộc:

```
/scan  
/odom  
/tf  
/map  
goal pose
```

Đầu ra bắt buộc:

```
/cmd_vel
```

Có thể test độc lập bằng RViz click goal.

Team E — Base / Hardware

Phụ trách:

```
clean_robot_base
```

Đầu vào bắt buộc:

```
/cmd_vel
```

Đầu ra bắt buộc:

```
/odom  
Robot thật chạy đúng
```

Team này có thể làm sau nếu giai đoạn đầu chỉ dùng Webots.

6. Thứ tự tích hợp

Không tích hợp kiểu một phát full system. Dễ vỡ và khó debug.

Giai đoạn 1 — Workspace và message

```
clean_robot_msgs build được.  
ros2 interface show chạy được.
```

Giai đoạn 2 — Simulation/Base chạy được

```
Robot nhận /cmd_vel và di chuyển.  
Camera/LiDAR/Odom publish topic ổn.
```

Giai đoạn 3 — Nav2 chạy được

```
RViz click goal → robot chạy tới.
```

Giai đoạn 4 — Mission chạy với fake target

```
fake_trash_candidate_node → target_manager → mission_manager → Nav2.
```

Giai đoạn 5 — Perception thật

```
camera → YOLO → detections → candidates.
```

Giai đoạn 6 — Full MVP

```
Robot tự tìm rác → thấy rác → chạy tới → dừng.
```

7. Quy tắc Git cho team

Branch

```
main
develop
feature/perception-yolo
feature/mission-manager
feature/nav2-config
feature/webots-sim
feature/robot-description
```

Không push thẳng vào `main`.

Flow:

```
feature/* → pull request → develop → main
```

Commit message

Dùng format:

```
<type>(<scope>): <message>
```

Ví dụ:

```
feat(perception): add trash detector skeleton
fix(mission): prevent target switching while navigating
docs(readme): update package responsibilities
config(nav2): tune local costmap params
```

Type gợi ý:

```
feat
fix
docs
config
refactor
test
```

8. Quy tắc code

Naming

Package:

```
clean_robot_<domain>
```

Node:

```
trash_detector_node  
mission_manager_node  
target_manager_node
```

Topic:

```
/trash/detections  
/trash/candidates  
/trash/target_pose  
/mission/state
```

Parameter file:

```
mission_params.yaml  
perception_params.yaml  
nav2_params.yaml
```

Logging

Mỗi node nên log rõ:

```
Node started  
Received input  
Published output  
State changed  
Goal sent  
Goal failed  
Goal succeeded
```

Ví dụ:

```
[mission_manager] State: SEARCHING → NAVIGATING_TO_TRASH  
[target_manager] Selected target trash_001 confidence=0.91
```

Không làm

```
Không publish /cmd_vel từ YOLO.  
Không hard-code topic lung tung.  
Không đổi message mà không báo team.  
Không gom perception + mission + nav2 vào một file.  
Không đẩy file build/, install/, log/ lên Git.
```

9. `.gitignore` gợi ý

```
build/  
install/  
log/  
__pycache__/  
*.pyc  
*.bag  
*.db3  
*.sqlite3  
.vscode/  
.idea/
```

10. Checklist trước khi merge

Một pull request chỉ nên merge khi:

- Build không lỗi
- Không phá topic contract
- Có hướng dẫn chạy/test
- Có log rõ ràng
- Không commit build/install/log
- Nếu đổi message, đã báo các team liên quan

Lệnh kiểm tra tối thiểu:

```
colcon build  
source install/setup.bash  
ros2 pkg list | grep clean_robot
```

11. Lệnh cài đặt cơ bản

Cài Webots:

```
sudo snap install webots
```

Cài ROS2 Jazzy:

```
sudo apt update  
sudo apt install ros-jazzy-desktop python3-argcomplete -y
```

Source ROS2:

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

Cài cầu nối Webots ROS2:

```
sudo apt install ros-jazzy-webots-ros2 -y
```

Kiểm tra:

```
ros2 --version  
ros2 topic list
```

12. Build workspace

```
cd ~/clean_robot_ws  
rosdep install --from-paths src -y --ignore-src  
colcon build  
source install/setup.bash
```

13. Chạy từng phần để debug

Kiểm tra message

```
ros2 interface show clean_robot_msgs/msg/TrashCandidate
```

Kiểm tra mission state

```
ros2 topic echo /mission/state
```

Kiểm tra target

```
ros2 topic echo /trash/target_pose
```

Kiểm tra Nav2 action

```
ros2 action list | grep navigate
```

Kiểm tra lệnh điều khiển xe

```
ros2 topic echo /cmd_vel
```

14. Roadmap MVP

Sprint 1

- Tạo workspace
- Tạo package clean_robot_msgs
- Tạo skeleton package
- Chốt topic contract

Sprint 2

- Webots robot chạy được bằng /cmd_vel
- Robot publish /camera/image_raw, /scan, /odom, /tf

Sprint 3

- Nav2 chạy được
- RViz click goal robot chạy tới

Sprint 4

- Mission Manager chạy với fake target
- Robot đi tới tọa độ rác giả

Sprint 5

- YOLO publish /trash/detections
- Localization publish /trash/candidates

Sprint 6

- Full MVP: tự tìm rác, thấy rác, chạy tới, dừng

15. Định nghĩa hoàn thành MVP

MVP được xem là hoàn thành khi:

1. Robot chạy trong Webots hoặc robot thật.
2. Camera publish ảnh.
3. Hệ thống tạo được /trash/candidates.
4. Target Manager chọn được /trash/target_pose.
5. Mission Manager gửi goal tới Nav2.
6. Nav2 publish /cmd_vel.
7. Robot chạy tới vị trí rác.
8. Robot dừng và /mission/state = REACHED_TRASH.

Chưa cần:

- Tay gấp
- Thùng rác
- Tự phân loại rác
- Tối ưu đường đi phức tạp